Finality Without Fork Choice: How Finalis Core Achieves Deterministic Settlement

Technical Whitepaper | April 2026 | Version 1.0 | reikagitakashi@gmail.com

Abstract

Finalis Core is a finalized-state BFT blockchain in which validator lifecycle, operator-native committee formation, adaptive checkpoint derivation, and future committee eligibility are all derived deterministically from finalized history. The node processes only `height = finalized_height + 1`. There is no live fork-choice path in the node runtime.

This paper describes the consensus protocol, deterministic committee derivation, economic model, light client RPC interface, and security properties of the system.

1. Introduction

1.1 The Problem

Most blockchains require a **live fork choice rule**:

Chain	Fork Choice Rule
Bitcoin	Longest chain (Nakamoto consensus)
Ethereum	GHOST (heaviest subtree)
Tendermint/Cosmos	Last finalized + vote extrapolation

These rules create complexity for:

- Wallet developers (reorg handling, confirmation heuristics)
- Exchange engineers (deposit finality, confirmation depth)
- Light clients (trusted headers, fraud proofs)
- Cross-chain bridges (finality gadgets, waiting periods)

1.2 The Alternative

Finalis Core eliminates fork choice entirely.

```
The node processes only:
    height = finalized_height + 1

No live chain selection. No "best chain" computation. No reorgs.
```

1.3 State Transition

The state transition is a pure function:

```
state(h+1) = Apply(finalized_block(h+1), state(h))
```

Where `finalized_block(h+1)` exists if and only if the network has produced a quorum certificate for that height.

1.4 Key Insight

In most chains, finality is an **emergent property** of fork choice + block depth. In Finalis Core, finality is **the only property** — fork choice is absent by design.

2. BFT Quorum Finality

2.1 Quorum Certificate (QC) Definition

Let:

- `N` = committee size at height `H`
- `Q(H)` = floor(2N/3) + 1 (BFT quorum threshold)
- `S(H,B)` = set of signatures from committee members for block `B` at height `H`

A block `B` at height `H` is **finalized** if:

```
finalized(H, B) := |S(H,B)| >= Q(H) ∧ valid_signatures(S(H,B))
```

Where `valid_signatures()` verifies:

- Each signature is from a distinct committee member
- Each signature is over `(H, round, block_id)`

- The signer was in the committee at height `H`

2.2 Voting Flow

```
Leader                  Validator 1          Validator 2          Validator N
   |                        |                    |                    |
   |   PROPOSE(H, B)        |                    |                    |
   |------------------------>|                    |                    |
   |   PROPOSE(H, B)        |                    |                    |
   |------------------------|--------------------->|                    |
   |   PROPOSE(H, B)        |                    |                    |
   |------------------------|--------------------|--------------------->|
   |                        |                    |                    |
   |                        |   VOTE(H, B)       |                    |
   |<-----------------------|                    |                    |
   |                        |   VOTE(H, B)       |                    |
   |<-----------------------|--------------------|                    |
   |                        |                    |                    |
   |                        |                    |   VOTE(H, B)       |
   |<-----------------------|--------------------|--------------------|
   |                        |                    |                    |
   |   QC(H, B)             |                    |                    |
   |------------------------>|                    |                    |
   |   QC(H, B)             |                    |                    |
   |------------------------|--------------------->|                    |
   |   QC(H, B)             |                    |                    |
   |------------------------|--------------------|--------------------->|
   |                        |                    |                    |
   ▼                        ▼                    ▼                    ▼
FINALIZED               FINALIZED            FINALIZED            FINALIZED
```

2.3 Safety Proof (No Forks)

Assume two distinct blocks `B1` and `B2` are finalized at the same height `H`:

```
|S(H,B1)| >= Q(H)  and  |S(H,B2)| >= Q(H)
```

By the pigeonhole principle, the intersection `S(H,B1) ∩ S(H,B2)` has size:

```
|intersection| >= 2*Q(H) - N
```

Substituting `Q(H) = floor(2N/3) + 1`:

```
2*(floor(2N/3) + 1) - N >= 2*(2N/3) - N + 2 = (4N/3 - N) + 2 = N/3 + 2
```

For `N >= 1`, `N/3 + 2 > 0`. Therefore, at least one honest validator would have signed two different blocks at the same height — equivocation. Honest validators never equivocate, so `B1 = B2`.

Therefore, no two distinct blocks can be finalized at the same height.

2.4 Liveness

If the network is synchronous and the leader is honest:

1. Leader proposes block `B`
2. `>= Q(H)` validators vote
3. Leader collects votes into QC
4. QC is broadcast, block is finalized

If the leader is faulty, timeout certificate triggers round change. Liveness holds with standard BFT assumptions (2/3 honest, synchronous after GST).

2.5 Restart Safety

On restart, the node:

1. Reads the last finalized height from disk
2. Derives the full canonical state by replaying finalized transitions
3. Begins consensus for `finalized_height + 1`

No "last finality certificate" ambiguity. No "what if we finalized two different blocks" — the BFT quorum prevents equivocation.

3. Deterministic Committee Formation

3.1 The Challenge

Most BFT chains have a live validator set that changes via external voting (on-chain governance, staking contract). Finalis Core derives committees **deterministically from finalized history** — no live votes for validator set changes.

3.2 Core Data Structures

```
struct FinalizedCommitteeCheckpoint {
    uint64_t epoch_start_height;
    Hash32 epoch_seed;
    uint8_t ticket_difficulty_bits;
    vector<PubKey32> ordered_members;
    vector<uint64_t> ordered_base_weights;
    vector<uint32_t> ordered_ticket_bonus_bps;
    DerivationMode mode;  // NORMAL or FALLBACK
    AdaptiveCheckpointParameters adaptive;
}

struct AdaptiveCheckpointParameters {
    uint64_t qualified_depth;          // Operators meeting min_bond + availability
    uint64_t target_committee_size;    // Derived from qualified_depth
    uint64_t min_eligible_operators;   // target + 3 (BFT safety margin)
    uint64_t min_bond;                 // Adjusts inversely with qualified_depth
    uint32_t target_expand_streak;     // Consecutive expansions
    uint32_t target_contract_streak;   // Consecutive contractions
}
```

3.3 Committee Derivation Flow

Step 1: Compute qualified_depth

```
qualified_depth = count_eligible_operators_at_checkpoint(
    validators,
    height,
    availability_state,
    availability_config_with_min_bond(availability_cfg, min_bond)
)
```

An operator is eligible if:

- Bonded amount >= min_bond
- Availability status is ACTIVE (not WARMUP/PROBATION/EJECTED)
- Passed liveness window checks

Step 2: Derive adaptive parameters

```
if (qualified_depth > target * 1.5):
    new_target = min(target * 2, qualified_depth)
    expand_streak += 1, contract_streak = 0
elif (qualified_depth < target * 0.75):
    new_target = max(target / 2, 1)
```

```
        contract_streak += 1, expand_streak = 0
    else:
        new_target = target
        expand_streak = 0, contract_streak = 0

    if (expand_streak > 3): new_target = min(new_target, qualified_depth)
    if (contract_streak > 2): new_target = max(new_target, qualified_depth / 2)
```

Step 3: Compute min_bond and min_eligible

```
min_eligible = target_committee_size + 3
base_bond = BOND_AMOUNT * (target_committee_size / 16)
depth_multiplier = max(1.0, qualified_depth / target_committee_size)
min_bond = base_bond / depth_multiplier
```

Step 4: Select committee members

```
candidates = []
for each eligible operator:
    weight = bond_amount * (1 + ticket_bonus_bps/10000)
    candidates.append(operator, weight)

sort(candidates, by weight descending)
committee = candidates[0:target]
```

3.4 Example Evolution

Epoch	Qualified Depth	Target	Min Bond	Mode
0 (genesis)	16	16	150 FLS	NORMAL
1	30	30	80 FLS	NORMAL
2	40	30 (stable)	80 FLS	NORMAL
3	10	8	300 FLS	NORMAL
4	6	8	300 FLS	NORMAL
5	5	8	300 FLS	FALLBACK

3.5 Fallback Mode

When `eligible_operator_count < min_eligible_operators`:

- Committee derivation switches to FALLBACK mode
- Availability enforcement is relaxed (`enforce_availability = false`)
- Hysteresis: recovery requires `eligible >= min_eligible + 1`
- Fallback mode is logged and exposed via RPC

This ensures the chain remains live even if operator participation drops below safety threshold.

4. Validator Lifecycle

4.1 Validator Status State Machine

```
enum class ValidatorStatus : uint8_t {
    PENDING = 0,    // Joined but in warmup
    ACTIVE = 1,     // Fully participating
    EXITING = 2,    // Unbonding, can't participate
    BANNED = 3,     // Slashed, permanently removed
    SUSPENDED = 4,  // Temporarily removed (liveness failure)
};
```

4.2 State Transitions

```
PENDING ──[warmup_blocks complete]──> ACTIVE
ACTIVE  ──[unbond request]──────────> EXITING
ACTIVE  ──[liveness failure]────────> SUSPENDED
SUSPENDED ──[suspend_duration pass]─> ACTIVE
ACTIVE/SUSPENDED ──[slashing]───────> BANNED
EXITING ──[cooldown_blocks pass]────> (removed from registry)
```

4.3 Liveness Tracking

Every block, validators in the committee are tracked:

```
info.eligible_count_window++      // Was in committee
if (participated)
    info.participated_count_window++
```

At window rollover (every 10,000 blocks by default):

```
miss_rate = (eligible - participated) * 100 / eligible

if (miss_rate >= 60%) → status = EXITING
if (miss_rate >= 30%) → status = SUSPENDED
// Reset counters for next window
```

4.4 Slashing

Slashing occurs for:

- Double signing (equivocation)
- Other Byzantine behavior

When slash evidence is found:

```
validators.ban(pub, height);           // Status → BANNED
validators.finalize_withdrawal(pub);   // Bond is forfeited
```

4.5 Operator Model

One operator (entity) can run multiple validators. Availability is tracked per-operator, not per-validator:

```
Operator "StakingHub" (operator_id = 0xABC...)
  ├── Validator Alice (pubkey 0x111...)
  └── Validator Bob   (pubkey 0x222...)

If Alice misses votes → Both Alice AND Bob are penalized
```

This prevents operators from running many unreliable validators.

5. Economic Model

5.1 Supply Cap

```
Total supply: 7,000,000 FLS (hard cap)
Emission: 12 years, 20% annual decay
Reserve accrual: 10% during emission era
Post-cap: fee pooling + reserve subsidy
```

5.2 Emission Schedule

```
Year 1: 1,000,000 FLS (base)
Year 2: 800,000 FLS (-20%)
Year 3: 640,000 FLS
Year 4: 512,000 FLS
Year 5: 409,600 FLS
Year 6: 327,680 FLS
Year 7: 262,144 FLS
Year 8: 209,715 FLS
Year 9: 167,772 FLS
Year 10: 134,218 FLS
Year 11: 107,374 FLS
Year 12: 85,899 FLS
After year 12: 0 FLS new issuance
```

Per-block reward:

```
reward_units(height) = emission_rate(height) / blocks_per_year
```

5.3 Reward Flow

```
                    BLOCK PRODUCED at height H
                              │
                              ▼
                    gross_reward = emission(H)
                              │
        ┌─────────────────────┼─────────────────────┐
        │                     │                     │
        ▼                     ▼                     ▼
  10% to Reserve        90% to Validators      Fees to Fee Pool
        │                     │                     │
        ▼                     ▼                     ▼
   Reserve Balance      Validator Reward      Fee Pool (post-cap)
   (post-cap subsidy)   = (90% * emission)/N   │
                        * (participation_bps)  │
                        + (fee_pool_share)     │
                                               │
                        ┌──────────────────────┘
                        ▼
                  Post-cap (after year 12):
                  Fee Pool → Validators
                  Reserve Subsidy → Fee Pool (if fees low)
```

5.4 Participation Penalty

Validators earn rewards proportional to their participation:

```
adjusted_score = raw_score * participation_bps / 10000

where participation_bps = min(observed_votes / expected_votes, 1) * 10000
```

Example:

```
expected_votes = 100 (validator in committee for 100 rounds)
observed_votes = 85
base_reward = 100 FLS

actual_reward = 100 × (85/100) = 85 FLS
```

5.5 Thresholds

Threshold	Value	Consequence
Miss rate ≥ 30%	Suspension	Temporarily removed from committee
Miss rate ≥ 60%	Exit	Unbond and leave validator set
Suspend duration	1,000 blocks	~2-3 hours (depending on block time)

5.6 Post-Cap Operation

After year 12 (or when 7M FLS is reached):

- No new coins are minted
- Fee pools are distributed to validators
- Reserve subsidy can supplement fee pools (drawn from accrued reserve)
- Validators earn only transaction fees + reserve subsidy (if any)

6. Lightserver RPC

6.1 Design Principle

All RPC methods read **only finalized state**. There is no "pending" or "mempool" view. This eliminates reorg risk for clients.

6.2 Core Methods

Method	Input	Output	Idempotent
`get_status`	`{}`	node status, tip height, availability	Yes
`get_tx_status`	`{txid}`	finalized status, height, credit safety	Yes
`get_tx`	`{txid}`	full transaction hex (if finalized)	Yes
`get_utxos`	`{scripthash, limit, start_after}`	paged UTXO list	Yes
`get_history_page`	`{scripthash, limit, start_after}`	paged transaction history	Yes
`validate_address`	`{address}`	validity, network match, scripthash	Yes
`broadcast_tx`	`{tx_hex}`	acceptance status (not finality)	No

6.3 Transaction Flow

```
Wallet/Client              Lightserver              Node/Mempool
      |                          |                        |
      |   broadcast_tx(tx_hex)   |                        |
      |-------------------------->|                        |
      |                          |   relay_tx(tx_hex)     |
      |                          |------------------------>|
      |                          |                        |
      |   {accepted: true}       |                        |
      |<--------------------------|                        |
      |                          |                        |
      |   (polling)              |                        |
      |   get_tx_status(txid)    |                        |
      |-------------------------->|                        |
      |                          |   query DB             |
      |                          |   (not yet finalized)  |
      |   {finalized: false}     |                        |
```

```
        |◀──────────────────────────┤                          |
        |                           |                          |
        |   ... later ...           |                          |
        |                           |                          |
        |   get_tx_status(txid)     |                          |
        ├──────────────────────────▶|                          |
        |                           |   query DB               |
        |                           |   (now finalized)        |
        |   {finalized: true,       |                          |
        |    height: 12345,         |                          |
        |    credit_safe: true}     |                          |
        |◀──────────────────────────┤                          |
        |                           |                          |
        ▼                           ▼                          ▼
  CREDIT SAFE                   (done)                     (done)
```

6.4 Credit Safety Rule

A transaction is credit safe if:

- It is finalized (`finalized = true`)
- Its height is <= `finalized_height - 1` (not the tip itself, to allow for rollback of unconfirmed children — though reorgs cannot happen, this is conservative)

6.5 Example RPC Responses

get_status response:

```
{
    "network_name": "mainnet",
    "finalized_height": 12345,
    "finalized_transition_hash": "0x...",
    "version": "0.1.0",
    "healthy_peer_count": 8,
    "latest_finality_committee_size": 16,
    "latest_finality_quorum_threshold": 11,
    "availability": {
        "epoch": 5,
        "eligible_operator_count": 12,
        "checkpoint_derivation_mode": "normal",
        "adaptive_regime": {
            "qualified_depth": 14,
            "adaptive_target_committee_size": 16,
            "adaptive_min_bond": 15000000000
        }
```

```
    }
}
```

get_tx_status response:

```
{
    "txid": "0x...",
    "status": "finalized",
    "finalized": true,
    "height": 12340,
    "finalized_depth": 6,
    "credit_safe": true,
    "transition_hash": "0x..."
}
```

get_utxos (paged) response:

```
{
    "items": [
        {"txid": "0x...", "vout": 0, "value": 100000000, "height": 12340}
    ],
    "has_more": true,
    "next_start_after": {"txid": "0x...", "vout": 1}
}
```

broadcast_tx response:

```
{
    "ok": true,
    "accepted": true,
    "txid": "0x...",
    "status": "accepted_for_relay",
    "retryable": false,
    "mempool_full": false
}
```

> **Important:** `accepted = true` means the transaction was accepted into the local mempool and will be relayed. It does **not** guarantee finalization. Clients must poll `get_tx_status` for settlement confirmation.

7. Security Properties

7.1 Safety Invariants

```
∀ heights H1, H2:
    finalized(H1) ∧ finalized(H2) ∧ H1 ≠ H2
    → H1 and H2 are on same chain
```

(No forks by construction)

7.2 Liveness Invariant

```
If network is synchronous and >2/3 of validators are honest,
then finalization eventually occurs for each height.
```

7.3 Deterministic State

```
If two nodes have processed the same set of finalized blocks,
they have identical state:
    state1 == state2
```

7.4 No Reorg Property

```
Once a block is finalized, it is never unfinalized:
    finalized(H, B) → ∀ future time t, finalized_t(H, B)
```

7.5 Attack Resilience

Attack	Mitigation
Equivocation	Slashing (bond forfeiture, banning)
Long-range fork	Impossible (no fork choice, finalized history only)
Sybil	Bond requirements, join window limiting
DoS (P2P)	Per-peer queues, rate limiting, timeouts
DoS (mempool)	Size limits (10MB, 10k txs), fee-based eviction
Nothing-at-stake	Bonded validation, slashing

8. Comparison with Other Chains

Feature	Bitcoin	Ethereum (post-merge)	Tendermint/Cosmos	Finalis Core
Fork choice	Longest chain	GHOST	Last finalized + extrapolation	None
Finality	Probabilistic	Probabilistic + finality gadget	Instant (BFT)	Instant (BFT)
Reorgs possible	Yes (deep)	Yes (shallow after finality)	No (under 1/3 faulty)	No (by construction)
Light client safety	Checkpoints	Sync committee	Trusted validator set	Finalized-state queries
Validator set changes	N/A	Live voting	Live staking	Deterministic from finalized history
Script language	Bitcoin Script	EVM (Turing-complete)	CosmWasm	Fixed P2PKH only
Supply cap	21,000,000 BTC	No fixed cap	Variable	7,000,000 FLS

9. Repository Structure

```
finalis-core/
├── src/
│   ├── consensus/        BFT, committees, checkpoints, TLA+ models
│   ├── node/             Node runtime, event loop, state machine
│   ├── lightserver/      JSON-RPC server, client library
│   ├── p2p/              Peer management, framing, handshake
│   ├── storage/          RocksDB interface, key prefixes
│   ├── utxo/             UTXO validation, script handling
│   ├── wallet/           Wallet logic, UTXO selection
│   ├── crypto/           Ed25519, hashing, Sparse Merkle Trees
│   └── mempool/          Transaction pool, fee policy
├── apps/
│   ├── finalis-node/       Node executable
│   ├── finalis-lightserver/ Lightserver executable
│   ├── finalis-explorer/   HTTP explorer (UI + API)
│   ├── finalis-wallet/     Qt desktop wallet
```

```
│   └── finalis-cli/        Command-line diagnostic tool
├── tests/
│   ├── test_consensus.cpp    Committee, finality, checkpoints
│   ├── test_lightserver.cpp  RPC client/server
│   ├── test_explorer.cpp     HTTP routes, pagination
│   ├── test_integration.cpp  Full node + wallet + explorer
│   └── test_utxo.cpp         UTXO validation, scripts
├── formal/                TLA+ models for critical components
└── scripts/               Attack simulations, helper scripts
```

8.2 Building and Running

```
# Install dependencies (Ubuntu/Debian)
apt update
apt install -y build-essential cmake ninja-build pkg-config libssl-dev \
  qtbase5-dev qtchooser qt5-qmake qtbase5-dev-tools \
  libsodium-dev librocksdb-dev curl jq

# Build
cmake -S . -B build -G Ninja
cmake --build build -j

# Run a node with lightserver
./build/finalis-node --db ~/.finalis/mainnet --with-lightserver

# Default ports:
# P2P: 19440
# Lightserver: 19444
# Explorer: 18080
```

8.3 Testing

```
# Run all tests
ctest --test-dir build --output-on-failure

# Run specific test
env FINALIS_TEST_FILTER=test_lightserver ./build/finalis-tests
```

10. Current Status and Roadmap

10.1 Current Status (April 2026)

- ✅ Canonical mainnet genesis defined
- ✅ Bootstrap testing active
- ✅ Genesis validator can start chain alone
- ✅ Chain finalizing under live test conditions
- ✅ Block timing enforced on intended schedule
- ✅ Node, wallet, explorer, lightserver, CLI consistent

10.2 Testing Sequence

The present test sequence is:

1. Bootstrap finalization
2. Reward settlement into spendable outputs
3. Normal transfer of coins to another participant
4. Standard post-genesis validator registration

10.3 Future Work

Area	Description
Performance	Benchmark TPS, latency under load
Security	Formal verification of TLA+ models
Tooling	Improve wallet, explorer, RPC documentation
Windows support	Packaging, installer, cross-platform builds
Documentation	Exchange integration guide, operator manual

11. References

11.1 Code Repository

```
https://github.com/finalis-core/finalis-core
```

11.2 Key Documentation

Document	Location
Consensus overview	`docs/CONSENSUS.md`
Live protocol spec	`docs/LIVE_PROTOCOL.md`
Checkpoint derivation	`docs/spec/CHECKPOINT_DERIVATION_SPEC.md`
Availability state completeness	`docs/spec/AVAILABILITY_STATE_COMPLETENESS.md`
Exchange integration	`docs/EXCHANGE_INTEGRATION.md`
Address format	`docs/ADDRESSES.md`

11.3 Formal Models

- TLA+ checkpoint models: `formal/`
- Availability models: `formal/availability/`
- Attack simulations: `scripts/protocol_attack_sim.py`

Appendix A: Quick Reference Card

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          FINALIS CORE REFERENCE                          │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│   One line:                                                              │
│   "A BFT blockchain where the node only processes finalized_height + 1"  │
│                                                                          │
│   Finality rule:                                                         │
│   |signatures| >= floor(2N/3) + 1                                        │
│                                                                          │
│   Committee formation:                                                   │
│   Derived deterministically from qualified operator depth                │
│                                                                          │
│   Light client:                                                          │
│   Any query to lightserver reads only finalized state                    │
│                                                                          │
│   Supply:                                                                │
│   7,000,000 FLS, 12-year emission, 20% decay, 10% reserve                │
│                                                                          │
│   Ports:                                                                 │
│   P2P: 19440, Lightserver: 19444, Explorer: 18080                        │
│                                                                          │
```

```
│   Repository:                                                          │
│   https://github.com/finalis-core/finalis-core                         │
│                                                                         │
│   License: MIT                                                          │
│                                                                         │
└─────────────────────────────────────────────────────────────────────────┘
```

Appendix B: Glossary

Term	Definition
BFT	Byzantine Fault Tolerance — system continues operating correctly even if some nodes behave maliciously
QC	Quorum Certificate — collection of signatures meeting the threshold `floor(2N/3) + 1`
Finalized state	State derived exclusively from blocks that have achieved BFT quorum
Checkpoint	Snapshot of committee composition at epoch boundaries
Epoch	Fixed number of blocks after which committee may change
Qualified depth	Number of operators meeting minimum bond and availability requirements
Fallback mode	Committee derivation mode when eligible operators fall below safety threshold
Credit safe	Transaction status indicating it is safe to credit (finalized and not at tip)
Lightserver	JSON-RPC service exposing only finalized-state queries
Operator	Entity that may run multiple validators; availability tracked per-operator

Document version: 1.0
Last updated: April 2026
Authors: Reikahi Takashi
License: MIT

Here is a concise section for your whitepaper explaining the mint:

Appendix D: Chaumian Mint Service (finalis-mint)

D.1 Overview

`finalis-mint` is a standalone Chaumian eCash mint service that operates alongside the Finalis Core consensus node. It enables **privacy-preserving digital cash** by issuing blinded notes backed by on-chain FLS reserves.

The service is **intentionally separate from consensus** — it uses the lightserver RPC to read finalized state and broadcast transactions, but does not participate in block production or validation.

D.2 How It Works

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                          MINT OPERATION FLOW                                 │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│   USER                                      MINT                             │
│     │                                         │                              │
│     │  1. Deposit FLS to reserve address      │                              │
│     ├──────────────────────────────────────────►│                              │
│     │                                         │                              │
│     │  2. Request blind signature             │                              │
│     │     (send blinded message)              │                              │
│     ├──────────────────────────────────────────►│                              │
│     │                                         │                              │
│     │  3. Receive signed blinded message      │                              │
│     ◄──────────────────────────────────────────┤                              │
│     │                                         │                              │
│     │  4. Unblind to obtain note              │                              │
│     │     (spendable digital cash)            │                              │
│     │                                         │                              │
│     │  ... off-chain transfers ...            │                              │
│     │                                         │                              │
│     │  5. Redeem note for FLS                 │                              │
│     ├──────────────────────────────────────────►│                              │
│     │                                         │                              │
│     │  6. Receive FLS to address              │                              │
│     ◄──────────────────────────────────────────┤                              │
│                                                                              │
└──────────────────────────────────────────────────────────────────────────────┘
```

D.3 Blind Signatures (Privacy Guarantee)

The mint uses RSA blind signatures, first proposed by David Chaum in 1983. The protocol ensures:

```
User:   blinded = message × r^e mod n
Mint:   signed_blinded = blinded^d mod n
User:   signature = signed_blinded / r mod n

Result: signature is a valid RSA signature on message
Property: Mint never learns message
```

Implication: The mint cannot link issuance to redemption. Each note is anonymous.

D.4 Reserve Management

The mint maintains an on-chain reserve wallet. Key policies:

Policy	Value	Purpose
`RESERVE_MAX_INPUTS`	8	Limit per redemption transaction
`RESERVE_CONSOLIDATE_UTXO_COUNT`	12	Trigger consolidation when fragmented
`RESERVE_AUTO_PAUSE_LOW_RESERVE`	10,000 FLS	Auto-pause redemptions when low
`RESERVE_EXHAUSTION_BUFFER`	10,000 FLS	Warn when reserve near exhaustion

Redemptions are processed only after the lightserver confirms finalization — no reorg risk.

D.5 Security Properties

Property	Implementation
Double-spend prevention	Persistent note state (spent flags)
Auditability	Signed reserve attestations via `/audit/export`
Operator authentication	HMAC-signed requests
Secret management	Pluggable backends (dir, env, command)
Retry resilience	Persistent delivery job queue for notifiers

D.6 Current Status

`finalis-mint` is a **development and testing tool**, not a production system. The README explicitly states:

> "It is a narrow scaffold, not a production mint: file-backed state, deterministic RSA blind-signing for development/testing, no federation, no multi-operator quorum custody."

D.7 Future Potential

With additional work, the mint could become a production privacy layer:

Required Change	Purpose
HSM or secure key store	Replace seed-derived RSA
Database replication	Replace single-file state
Rate limiting	Prevent DoS attacks
TLS for all endpoints	Secure communication
N-of-M threshold signatures	Federation (multi-operator)

D.8 Relationship to Consensus

The mint **depends on** Finalis Core for:

- Finalized state reads (deposit confirmation)
- Transaction broadcast (redemptions)
- Reserve wallet management

The mint **does not affect** consensus:

- No mint transactions enter the mempool (except redemptions)
- The mint has no special privileges
- The chain operates identically with or without the mint

Summary: `finalis-mint` is a Chaumian eCash service that demonstrates privacy-preserving digital cash on top of Finalis Core. It is currently a development tool, designed as a foundation for future production privacy layers.

This whitepaper is provided for informational purposes only. The Finalis Core project is in active development; specifications may change. Always refer to the latest code and documentation.